

AI ASSISTED CODING

ASSIGNMENT : 10.2

PROGRAM : B.TECH\CSE
SPECIALISATION : AIML
NAME : MANDALA.TEJASRI
ENROLMENT NUMBER : 2403A51171
COURSE : AI ASSISTED CODING
DATE : 07-10-2025
BATCH NO : 01

TASK DESCRIPTION#1

>>Use google gemini to use an AI assistant to review and suggest corrections of the given code

PROMPT :

>>Develop a python code by using an AI assistant to review and suggest corrections of the given code

GIVEN CODE :

```

def calcFact(n):
    result=1
    x=0
    for i in range(1,n):
        result=result*i
    return result

def main():
    num = 5
    FACT = calcFact(num)
    print("the factorial of",num,"is",FACT)
    t=10
    if FACT>10:
        print("BIG Number")
    else:
        print("small number")

main()

```

CORRECTED CODE :

```

def calcFact(n):
    # Initialize result to 1, as the factorial of 0 is 1, and it's the multiplicative identity.
    result = 1
    # x = 0 is unnecessary and has been removed.

    # The loop should include 'n' itself in the multiplication.
    # range(1, n + 1) iterates from 1 up to and including n.
    for i in range(1, n + 1):
        result = result * i
    # The line 'result = result * i' must be INSIDE the for loop, and is now correctly placed.

```

```
def main():
    num = 5
    FACT = calcFact(num)

    # Output the result
    print("the factorial of", num, "is", FACT)

    # The variable 't' is not used and has been removed.

    # Conditional check
    if FACT > 10:
        print("BIG Number")
```

```
        print("BIG Number")
    else: # The missing colon (:) has been added here.
        print("small number")

    Call the main function to start the program execution
    main()
```

EXPECTED OUTPUT :

```
the factorial of 5 is 120
BIG Number
```

OBSERVATION :

Based on the **corrected** version of the Python code (as the original image had errors), here is the observation and predicted output:

1. Program Execution Flow

1. The program starts by calling `main()`.
2. Inside `main()`, the variable `num` is set to 5.
3. The `calcFact(5)` function is called.
 - `result` is initialized to 1.
 - The `for` loop runs from $i = 1$ up to 5 (inclusive).

- The calculation performs:
 - $i = 1$: `result` becomes $1 \times 1 = 1$
 - $i = 2$: `result` becomes $1 \times 2 = 2$
 - $i = 3$: `result` becomes $2 \times 3 = 6$
 - $i = 4$: `result` becomes $6 \times 4 = 24$
 - $i = 5$: `result` becomes $24 \times 5 = 120$
- The function returns **120**.

Back in `main()`, `FACT` is assigned the value **120**.

4. Back in `main()`, `FACT` is assigned the value **120**.
5. The first `print` statement executes.
6. The `if` condition checks if **120 > 10**. This is **True**.
7. The code block under the `if` statement executes.

TASK DESCRIPTION#2

>>Use google gemini to improve variable names, add comments, and apply PEP8 formatting of the given code

PROMPT :

>>Develop a python code to improve variable names, add comments, and apply PEP8 formatting of the given code

GIVEN CODE:

```
def f1(xX):
    a=0
    b=1
    c=2
    Zz=[a,b]
    while c<=xX:
        d=a+b
        Zz.append(d)
        a=b
        b=d
        c=c+1
    return Zz

def m():
    NN=10
    ans=f1(NN)
    print("fib series till",NN,":",ans)

m()
```

CORRECTED CODE AND EXPECTED OUTPUT :

```
def generate_fibonacci(limit):
    """
    Generates the Fibonacci series up to a specified number of terms.

    Args:
        limit (int): The number of terms to generate in the series.

    Returns:
        list: A list containing the Fibonacci series.
    """
    # Initialize the first two Fibonacci numbers
    # a represents the (n-2)th term
    # b represents the (n-1)th term
    a = 0
    b = 1

    # Initialize the series list with the first two terms
    fib_series = [a, b]

    # Initialize the counter for the loop. We already have 2 terms.
    counter = 2
```

```

# Initialize the first two Fibonacci numbers
# 'a' represents the (n-2)th term
# 'b' represents the (n-1)th term
a = 0
b = 1

# Initialize the series list with the first two terms
fib_series = [a, b]

# Initialize the counter for the loop. We already have 2 terms.
counter = 2
```

```
def main():
    """
    Main function to execute the Fibonacci series generation.
    """
    # Set the desired number of terms for the series
    num_terms = 10

    # Call the function and store the result
    result = generate_fibonacci(num_terms)

    # Print the final result in a clear format
    print(f"Fibonacci series till {num_terms} terms: {result}")
```

OBSERVATION :

Here is the observation of its execution and predicted output:

1. Execution Flow

1. The `if __name__ == "__main__":` block calls the `main()` function.
2. Inside `main()`, the variable `num_terms` is set to **10**.
3. The `generate_fibonacci(10)` function is called.
 - The variables `a` (0) and `b` (1) are initialized.
 - The list `fib_series` is initialized to `[0, 1]`.
 - The `counter` starts at 2.

- The `while counter < 10` loop runs for 8 iterations (when `counter` is 2, 3, 4, 5, 6, 7, 8, 9).

Counter	<i>a</i>	<i>b</i>	next_term (<i>a</i> + <i>b</i>)	fib_series (at end of loop)
2	0	1	1	[0, 1, 1]
3	1	1	2	[0, 1, 1, 2]
4	1	2	3	[0, 1, 1, 2, 3]
5	2	3	5	[0, 1, 1, 2, 3, 5]
6	3	5	8	[0, 1, 1, 2, 3, 5, 8]

5. The function returns the final list: `[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]` .
6. The `main()` function uses an **f-string** to print the result, clearly stating the number of terms and the generated list.

2. Predicted Output (Observation)

The code will print the following single line to the console:

```
Fibonacci series till 10 terms: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Task Description#3

>>use google gemini to Write a Python script with 3–4 functions (e.g., calculator: add, subtract, multiply, divide) and Incorporate manual docstring in code with NumPy Style to generate a module-level docstring + individual function

PROMPT:

Develop a python script with 3–4 functions (e.g., calculator: add, subtract, multiply, divide) and Incorporate manual docstring in code with NumPy Style to generate a module-level docstring + individual function

EXPECTED OUTPUT :

def add(a, b):

"""

Calculates the sum of two numbers.

Parameters

a : int or float

The first number (addend).

b : int or float

The second number (addend).

Returns

int or float

The sum of a and b.

Examples

```
>>> add(5, 3)
```

```
8
```

```
>>> add(10.5, 2.5)
```

```
13.0
```

```
.....
```

```
return a + b
```

```
def subtract(a, b):
```

```
.....
```

Calculates the difference between two numbers.

Parameters

a : int or float

The number to subtract from (minuend).

b : int or float

The number to subtract (subtrahend).

Returns

int or float

The result of a minus b.

Examples

```
>>> subtract(10, 4)
```

```
6
```

```
>>> subtract(20.5, 5.0)
```

```
15.5
```

```
"""
```

```
return a - b
```

```
def multiply(a, b):
```

```
    """
```

```
    Calculates the product of two numbers.
```

Parameters

a : int or float

The first factor.

b : int or float

The second factor.

Returns

int or float

The product of a and b.

Examples

```
>>> multiply(6, 7)
```

42

```
>>> multiply(3.0, 1.5)
```

4.5

.....

```
return a * b
```

```
def divide(a, b):
```

.....

Calculates the quotient of two numbers.

Parameters

a : int or float

The numerator (dividend).

b : int or float

The denominator (divisor).

Returns

int or float

The result of a divided by b.

Raises

ZeroDivisionError

If the denominator b is zero.

Examples

```
>>> divide(10, 2)
```

```
5.0
```

```
>>> divide(10, 3)
```

```
3.3333333333333335
```

.....

```
if b == 0:
```

```
    raise ZeroDivisionError("Cannot divide by zero.")
```

```
return a / b
```

Example usage of the functions (optional, but useful for testing)

```
if __name__ == "__main__":
```

```
    x = 10
```

```
    y = 5
```

```
    print(f"Addition: {x} + {y} = {add(x, y)}")
```

```
    print(f"Subtraction: {x} - {y} = {subtract(x, y)}")
```

```
    print(f"Multiplication: {x} * {y} = {multiply(x, y)}")
```

```
    print(f"Division: {x} / {y} = {divide(x, y)}")
```

```
# Example of raising an error

# try:

#     divide(x, 0)

# except ZeroDivisionError as e:

#     print(f"Error: {e}")
```

OBSERVATION:

1. Structure and Docstrings

The code is structured as a **Python module** intended for basic arithmetic. It effectively utilizes **four distinct functions** (`add`, `subtract`, `multiply`, `divide`), fulfilling the requirement of the prompt.

- **Module-Level Docstring:** A comprehensive docstring is present at the start, clearly defining the module's purpose and scope.
- **Function Docstrings (NumPy Style):** Each function incorporates detailed docstrings in the **NumPy format**, including sections for `Parameters`, `Returns`, and `Examples`. The `divide` function correctly includes a `Raises` section for the `ZeroDivisionError`. This

2. Execution Logic and Functionality

The script demonstrates the correct arithmetic functionality for each operation:

- **Inputs:** The main execution block (`if __name__ == "__main__":`) sets the input variables as `x = 10` and `y = 5`.

- **Calculations:**
 - `add(10, 5)` → **15**
 - `subtract(10, 5)` → **5**
 - `multiply(10, 5)` → **50**
 - `divide(10, 5)` → **2.0** (The result is a float due to true division `/`).
- **Error Handling (Implicit):** Although not explicitly executed in the final output, the `divide` function correctly includes logic to prevent division by zero using `if b == 0: raise ZeroDivisionError`.

3. Predicted Output

When the script is executed directly (`if __name__ == "__main__":`), it calls all four functions with $x = 10$ and $y = 5$ and prints the results using f-strings.

The script will print the following output:

```
Addition: 10 + 5 = 15
Subtraction: 10 - 5 = 5
Multiplication: 10 * 5 = 50
Division: 10 / 5 = 2.0
```